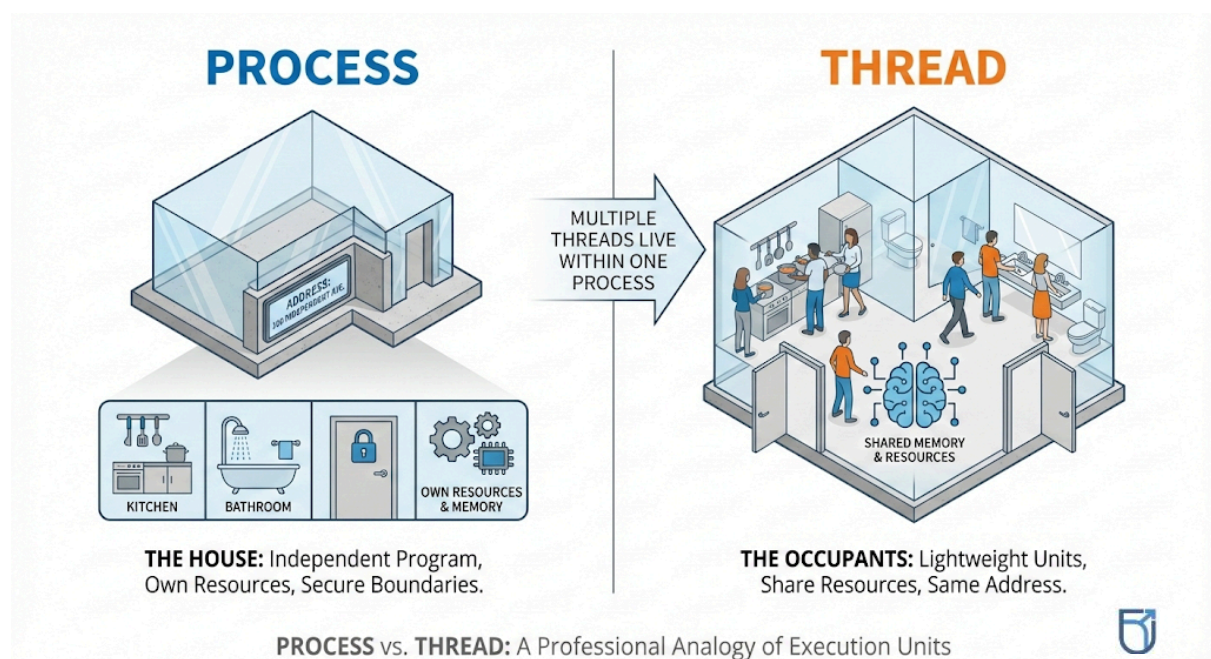


Lecture 04 NOTES – Threads in Depth: Models and Implementations

Process vs. Thread:

1. **Process (The House)** A **Process** is a completely independent program in execution. Think of it as a **house**. It has its own address, its own locked doors (security), and its own resources (kitchen, bathroom).

2. **Thread (The Occupants)** A **Thread** is a lightweight unit of execution *inside* a process. Think of threads as the **people living inside that house**. They all share the same address and the same resources (they use the same kitchen and bathroom).



Feature	Process (Heavyweight)	Thread (Lightweight)

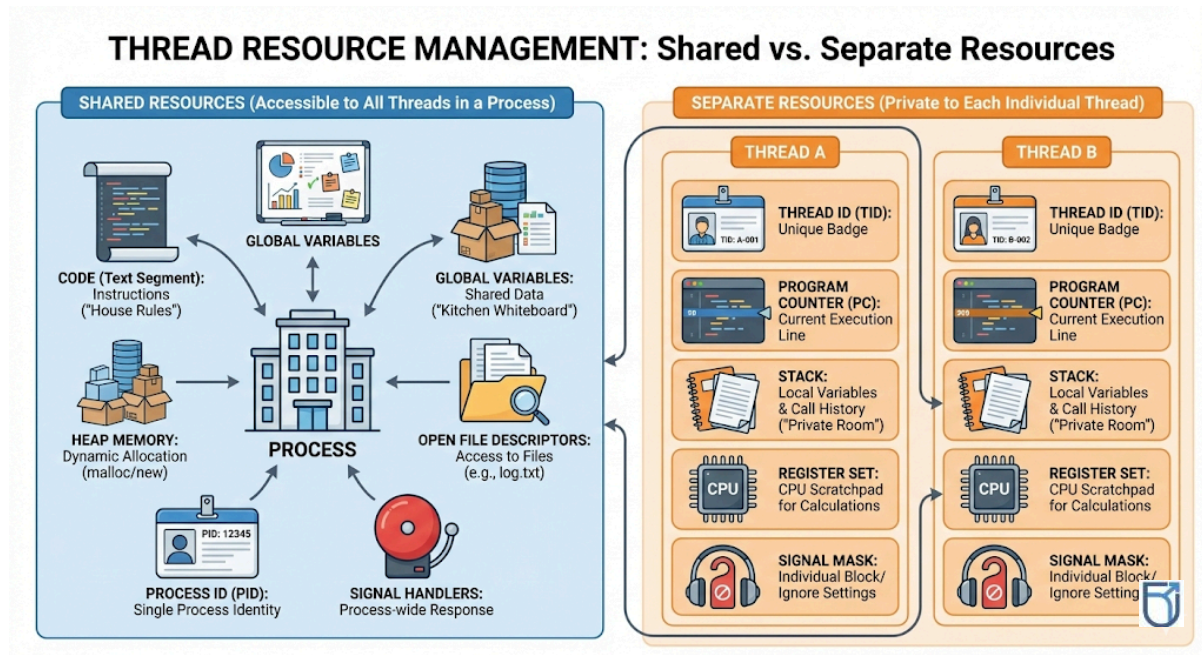
Definition	An independent program in execution.	A segment of a process; a path of execution.
Memory	Has its own isolated address space.	Shares the address space of the parent process.
Resources	Owens resources (files, I/O).	Shares resources (code, data, files) with other threads.
Creation Cost	Expensive: Requires new memory & PCB setup.	Cheap: Only needs a stack and register set.
Communication	Difficult: Requires IPC (pipes, sockets).	Easy: Can communicate directly via shared memory .
Isolation	High: If one process crashes, others are safe.	Low: If one thread crashes, the entire process may crash.
Context Switch	Slow: Involves saving heavy state info.	Fast: Minimal state saving required.
Example	Opening Microsoft Word and Spotify separately.	Microsoft Word auto-saving while you type (2 threads).

Thread Resource Management

1. Shared Resources

These are the resources that **all threads** in the process can access.

- **Code (Text Segment):** The actual instructions of the program. Everyone follows the same "house rules."
- **Global Variables (Data Segment):** Data declared outside of functions. It's like a whiteboard in the kitchen—anyone can read or write to it.
- **Heap Memory:** Dynamic memory allocated (using `malloc` or `new`). If one thread creates an object here, others can access it if they have the pointer.
- **Open File Descriptors:** If one thread opens a file (e.g., `log.txt`), all other threads can read/write to it immediately.
- **Process ID (PID):** The entire group shares the same "address" or identity number.
- **Signal Handlers:** If the process gets a "terminate" signal, it affects everyone.



2. Separate Resources (The "Private Rooms")

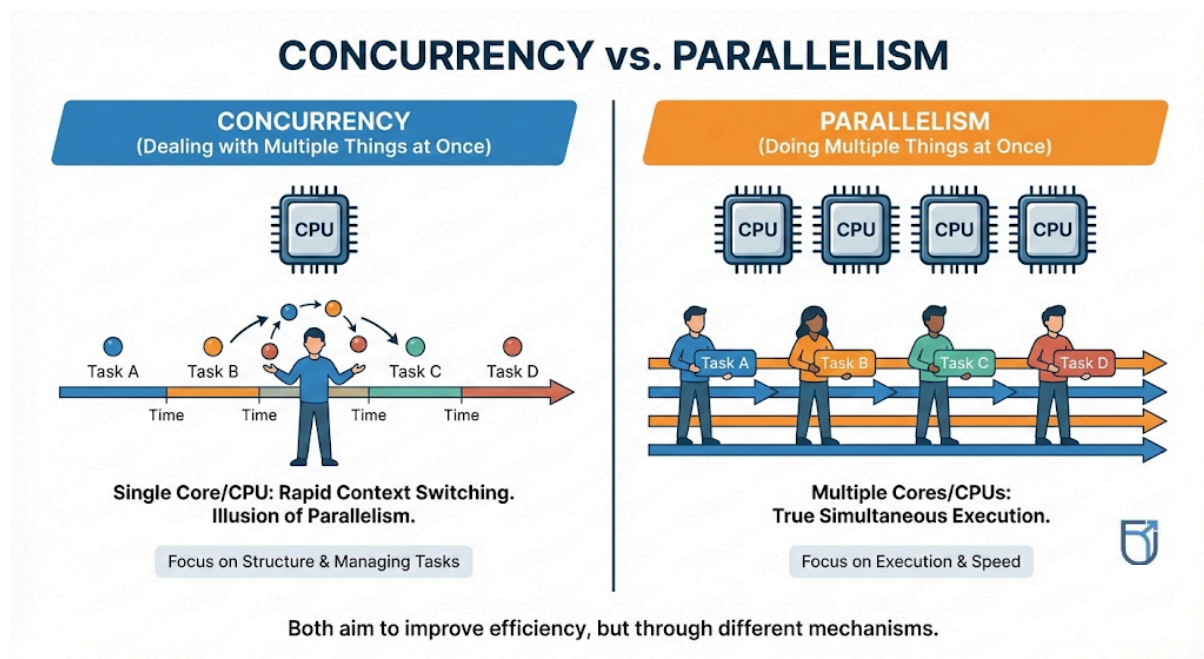
These are specific to **each individual thread**.

- **Thread ID (TID):** Each thread has its own unique badge number inside the process.
- **Program Counter (PC):** Tracks **exactly which line of code** this specific thread is executing right now. One thread might be on line 50, while another is on line 200.
- **Stack:** This is the most critical private area. It holds **local variables** and function call history. If Thread A calls `functionX()`, its local variables must not mix with Thread B calling `functionY()`.
- **Register Set:** The "scratchpad" for the CPU. Each thread needs its own working values to resume calculation after a context switch.
- **Signal Mask:** Threads can decide individually which signals they want to block (ignore).

Concurrency vs. Parallelism

1. Concurrency (The Structure)

- **Focus: Dealing** with multiple things at once.
- **Mechanism:** It is about the **logic** of the program. A single CPU switches between tasks (time-slicing) so fast that they *seem* to run together.
- **Key Requirement:** Can happen on a **Single Core**.



2. Parallelism (The Execution)

- **Focus: Doing** multiple things at once.
- **Mechanism:** It is about the **hardware** capability. Tasks physically run at the exact same nanosecond on different processors.
- **Key Requirement:** Requires **Multi-Core CPUs**.

Feature	Concurrency	Parallelism
Core Definition	The ability of a system to handle multiple tasks at once.	The ability of a system to execute multiple tasks simultaneously.
The “Mantra”	It is about Structure . (Designing a program to be decomposable).	It is about Execution . (Actually running things at the same time).

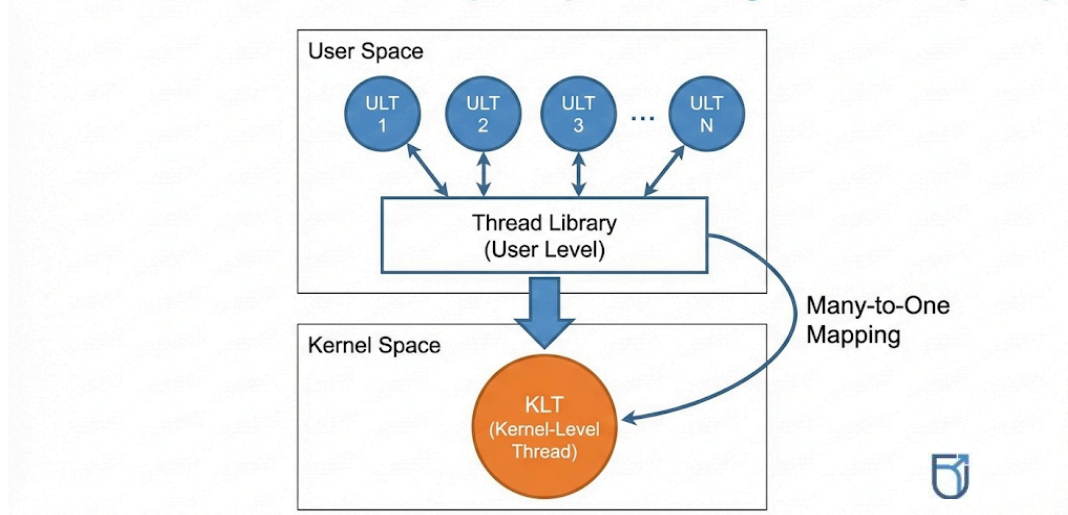
Hardware Requirement	Can run on a single core (or multiple).	Requires multiple cores (or multiple CPUs).
Mechanism	Interleaving: The CPU switches between tasks rapidly (context switching).	Simultaneous: Multiple tasks run on separate cores at the exact same instant.
The Goal	To maximize CPU utilization and minimize waiting time (hiding latency).	To maximize throughput and computational speed (crunching numbers faster).
Control Flow	Tasks have overlapping lifetimes but do not necessarily run at the same instant.	Tasks literally run at the same time.
Overhead	High: Due to frequent context switching and cache flushing.	Low: Less switching, but requires overhead for synchronization and communication.
Debugging	Difficult: Nondeterministic behavior makes bugs hard to reproduce (Heisenbugs).	Complex: Issues arise from race conditions and memory consistency models.
Best Use Case	I/O-bound tasks: Web servers, UIs, database queries (waiting for network/disk).	CPU-bound tasks: Video rendering, matrix multiplication, scientific simulations.
Real-World Analogy	One ATM machine with two lines; the machine serves one person from Line A, then one from Line B.	Two ATM machines serving two lines independently at the same time.

User-Level Threads (ULT) — Many-to-One (N:1)

1. Concept

- **Managed in User Space:** These threads are created and managed entirely by the application (user), not by the Operating System (OS).
- **Kernel Unaware:** The OS kernel does not know these threads exist. It thinks the process is just a single normal program.
- **N:1 Mapping:** "Many-to-One" means **many** user-level threads are mapped to just **one** kernel thread.
- **Green Threads:** They are sometimes nicknamed "green threads."

User-Level Threads (ULT) — Many-to-One (N:1)



2. Mechanics (How it Works)

- **Thread Library:** A library (code inside your program) manages everything. It creates and stores **Thread Control Blocks (TCBs)** to keep track of each thread.
- **User Space Switching:** When switching from thread A to thread B, it happens inside the user space.
- **No System Calls:** The program doesn't need to ask the OS to switch threads. This avoids slow system calls.
- **Super Fast:** Because the OS isn't involved, switching takes only **nanoseconds** (extremely fast).

3. Advantages (Pros)

- **Fast Creation/Destruction:** Making a new thread or deleting one is very quick because no kernel resources are used.
- **Fast Context Switching:** Jumping between threads is much faster than kernel-level threads.
- **High Scalability:** You can create **millions** of threads without slowing down the system.

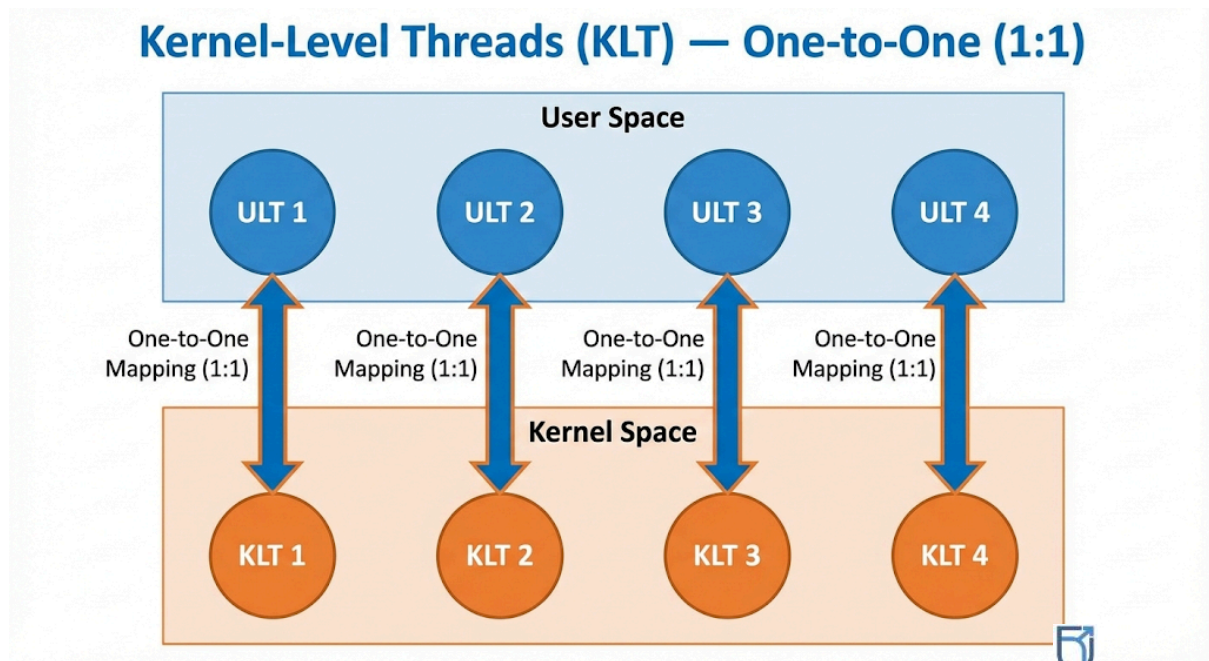
4. Limitations (Cons)

- **Blocking Issues:** If one thread executes a blocking system call (like waiting for input), the **entire process stops**. The OS sees the whole process as "waiting" and pauses everything.
- **No True Parallelism:** You cannot run threads on different CPU cores at the same time. Since the OS sees only one process, it assigns it to only one core.
- **Page Faults:** If a "page fault" occurs (memory access issue), the OS blocks the entire process, stopping all threads.

Kernel-Level Threads (KLT) — One-to-One (1:1)

1. Concept

- **OS Managed:** These threads are managed directly by the Operating System (Kernel), not by the application.
- **Kernel Aware:** The Kernel knows about every single thread created.
- **1:1 Mapping:** Each User Thread is connected to exactly **one** Kernel Thread.
- **Native Threads:** Often called "native threads" (e.g., POSIX Pthreads on Linux, Windows threads).



2. Mechanics

- **System Calls:** Creating a thread requires making a "System Call" to the OS (requests kernel intervention).
- **Kernel TCBs:** The Kernel maintains the **Thread Control Block (TCB)** for every thread in its own memory space.
- **Kernel Scheduling:** The Kernel's scheduler decides which thread runs on the CPU.
- **Mode Switch:** Context switching is slower because it requires a full "Mode Switch" (jumping from User Mode to Kernel Mode and back).

3. Advantages

- **True Parallelism:** Threads can run on different CPU cores simultaneously.
- **Independent Blocking:** If one thread blocks (e.g., waiting for I/O), the Kernel can schedule **other threads** from the same process to run. The process does not freeze.
- **Multicore Utilization:** Fully utilizes modern multi-core processors.

4. Disadvantages

- **High Overhead:** Creating and destroying threads is expensive (slow) because of system calls.
- **Slower Context Switching:** Switching between threads takes more time than User-Level threads due to the mode switch.
- **Limited Scalability:** You cannot create millions of threads; usually limited to thousands because each one consumes Kernel memory.

Performance Comparison (ULT vs KLT)

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Thread Creation	Faster (Managed in user space, no system calls)	Slower (Requires system calls to OS)
Context Switching	Faster (No mode switch needed)	Slower (Requires full mode switch to kernel)
Parallelism	None (Cannot run on multiple CPUs simultaneously)	True Parallelism (Can run on multiple CPUs simultaneously)
Blocking	Blocks Entire Process (If one thread blocks, all stop)	Blocks One Thread (Other threads continue running)

Scalability	High (Can support millions of threads)	Limited (Limited by kernel memory, usually thousands)
--------------------	---	--

Hybrid Threads — Many-to-Many (M:N)

1. Concept

- **Best of Both Worlds:** This model tries to combine the speed of User-Level Threads (ULT) with the parallelism of Kernel-Level Threads (KLT).
- **M:N Mapping:** A large number of **User Threads (M)** are mapped to a smaller or equal number of **Kernel Threads (N)**.
- **Two-Level Scheduling:**
 1. **User Scheduler:** A user-level library decides which User Thread runs on which available Kernel Thread.
 2. **Kernel Scheduler:** The OS Kernel decides which Kernel Thread runs on the actual CPU core.

2. Advantages

- **High Concurrency:** You can create as many user threads as you need (like ULT).
- **True Parallelism:** Because there are multiple kernel threads available, the system can run tasks on multiple processors at the same time (like KLT).
- **Better I/O Handling:** If one user thread performs a blocking I/O call, it only blocks the specific kernel thread it is using. The other kernel threads can continue running other user threads.

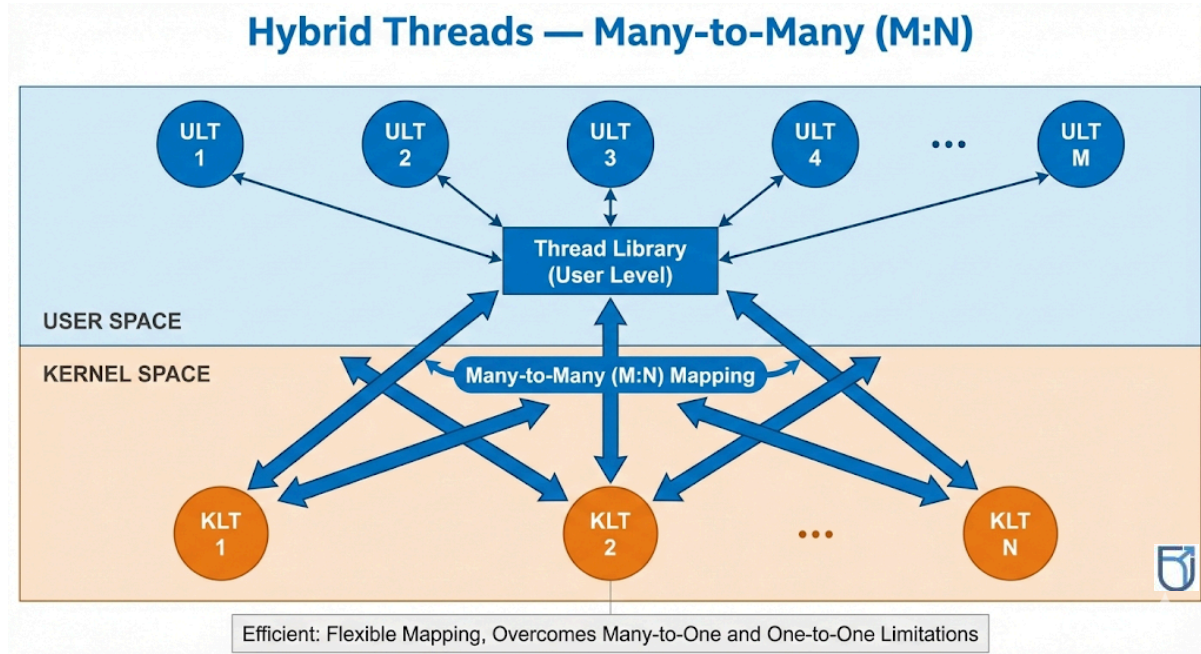
3. Disadvantages

- **Very Complex:** Implementing this system is extremely difficult because the user library and the kernel need to communicate efficiently.
- **Hard to Debug:** Identifying bugs is tricky because problems could arise from the user scheduler, the kernel scheduler, or the interaction between them.
- **Scheduling Overhead:** Managing two different layers of scheduling adds extra processing work (overhead) to the system.

Virtual Threads (Java 21+ / Project Loom)

Concept

- **Production-Ready M:N:** This is a modern, successful implementation of the "Many-to-Many" (Hybrid) model. It fixes the complexity issues of older attempts.
- **Lightweight:** You can create **millions** of virtual threads on a standard machine (unlike Kernel threads, which are limited to thousands).
- **Best of Both Worlds:** It allows developers to write simple "blocking-style" code (easy to read) but get the high performance of asynchronous code.



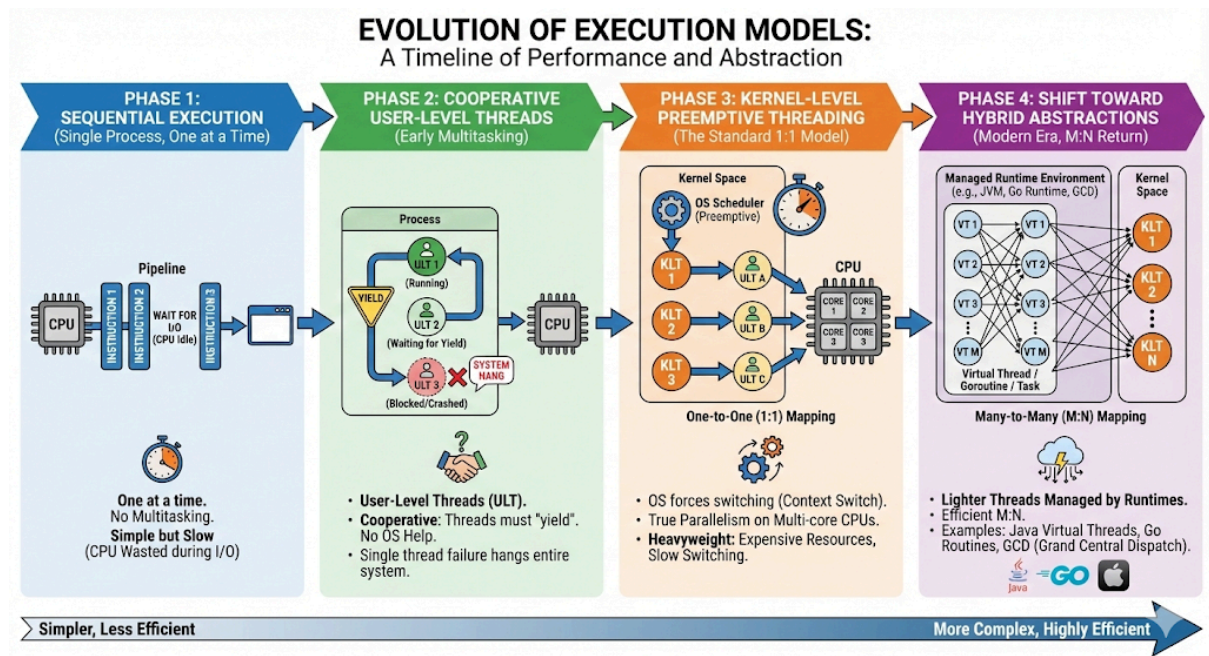
Mechanics

- **Carrier Threads:** Virtual threads run on top of a small pool of Kernel Threads, called "Carrier Threads."
- **Mounting:** When a virtual thread needs to do work (calculate something), it "mounts" (attaches) itself to a Carrier Thread.
- **Unmounting (The Magic):** When a virtual thread performs a blocking operation (like waiting for data from a database):
 1. It "unmounts" (detaches) from the Carrier Thread.
 2. The Carrier Thread is immediately free to run a *different* virtual thread.
- **Heap Storage:** While waiting, the virtual thread is stored in the **Heap memory** (cheap user memory). It does not hold onto the heavy OS thread resources.

Why it matters

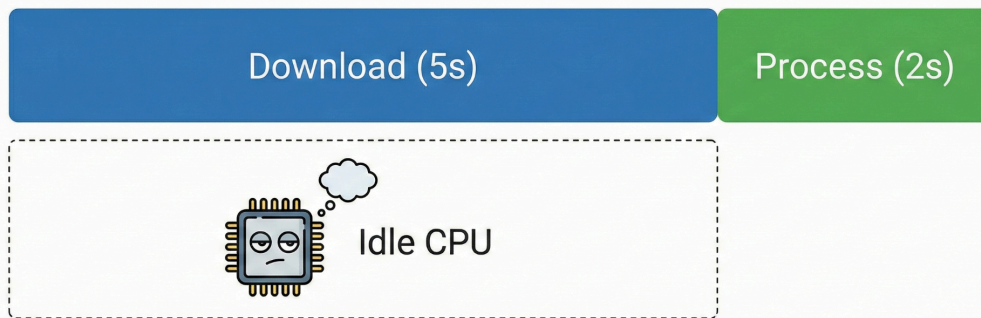
- It solves the "Fatal Flaw" of User-Level threads because a blocking call **does not block the underlying kernel thread**.
- It makes building high-throughput servers (like web servers) much easier and cheaper.

Evolution of Execution Models



Phase 1: Sequential Execution

- **One at a time:** The CPU executed one instruction after another for a single process.
- **No Multitasking:** If one program was running, nothing else could run until it finished.
- **Simple but Slow:** Very easy to understand, but wasted a lot of CPU time waiting for I/O.



CPU is bored while waiting for the internet.

Implementation:

```
#include <stdio.h>
#include <unistd.h>

void downloadFile(char* filename) {
    printf("Starting download: %s\n", filename);
    sleep(5); // Blocks for 5 seconds
    printf("Download complete: %s\n", filename);
}

void processFile(char* filename) {
    printf("Processing: %s\n", filename);
    sleep(2); // Runs for 2 seconds
    printf("Processing complete: %s\n", filename);
}

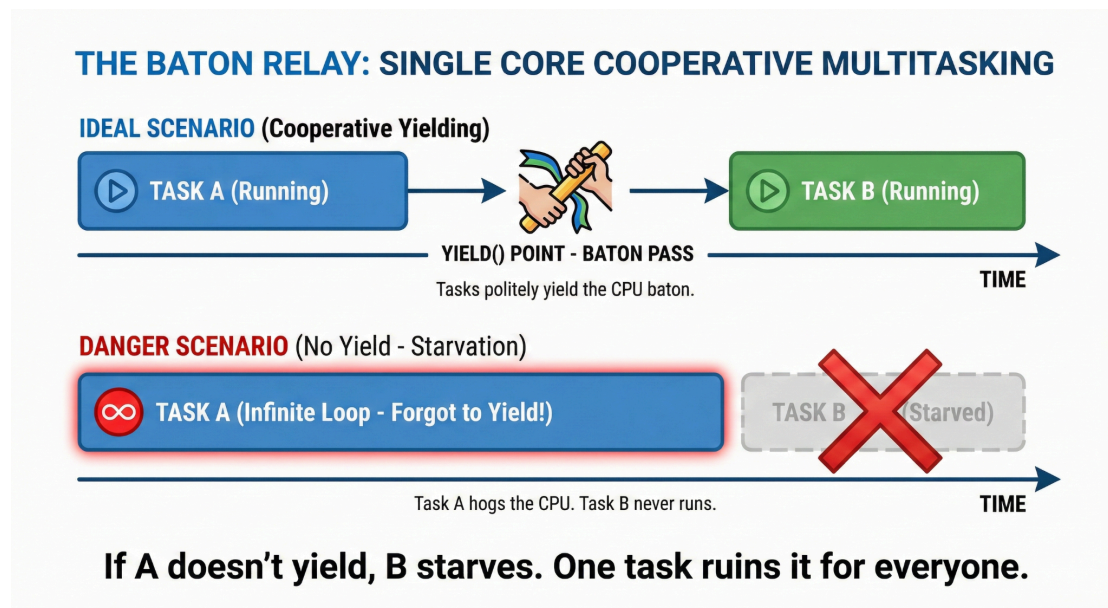
int main() {
    downloadFile("minishell.txt"); // Blocks for 5s
    processFile("A.txt");          // Runs for 2s
    // Total Time: 7s. CPU is idle during download.
    printf("Total time: 7s\n");
    return 0;
}
```

Output :

Starting download: minishell.txt
Download complete: minishell.txt
Processing: A.txt
Processing complete: A.txt
Total time: 7s

Phase 2: Cooperative User-Level Threads

- **Early Multitasking:** Programs used User-Level Threads (ULT).
- **"Cooperative":** The threads had to strictly "yield" control. If one thread refused to give up the CPU (or crashed), the entire system hung.
- **No OS Help:** The OS didn't help with switching or protection.



Implementation:

```
#include <stdio.h>
#include <unistd.h>

/* Task states */
typedef enum {
    READY,
    WAITING,
    DONE
} State;

/* Task control block */
typedef struct {
```

```

    const char *name;
    State state;
    int step;
} Task;

/* Two tasks */
Task taskA = {"TaskA", READY, 0};
Task taskB = {"TaskB", READY, 0};

/* Yield function (user-space context switch) */
void yield() {
    printf(">> Yielding CPU to another task ... \n\n");
}

/* Simulated I/O */
void startDownload(const char *file) {
    printf("Downloading %s ... \n", file);
}

/* Simulated processing */
void processFile(const char *file) {
    printf("Processing %s\n", file);
}

/* Task A */
void runTaskA(Task *t) {
    if (t->step == 0) {
        startDownload("A.txt");
        t->state = WAITING;
        t->step++;
        yield();
    }
    else if (t->step == 1) {
        printf("Download finished for A.txt\n");
        processFile("A.txt");
        t->state = DONE;
    }
}

```

```

/* Task B */
void runTaskB(Task *t) {
    if (t->step == 0) {
        printf("Task B: Calculating math... \n");
        sleep(1);
        printf("Task B: Calculation done\n");
        t->state = DONE;
    }
}

/* Simple cooperative scheduler */
int main() {
    while (taskA.state != DONE || taskB.state != DONE) {

        if (taskA.state == READY) {
            runTaskA(&taskA);
        }

        if (taskB.state == READY) {
            runTaskB(&taskB);
        }

        if (taskA.state == WAITING) {
            sleep(2);          // Simulate download delay
            taskA.state = READY;
        }
    }

    printf("\nAll tasks completed.\n");
    return 0;
}

```

Output :

```

Downloading A.txt...
>> Yielding CPU to another task...

Task B: Calculating math...
Task B: Calculation done
Download finished for A.txt

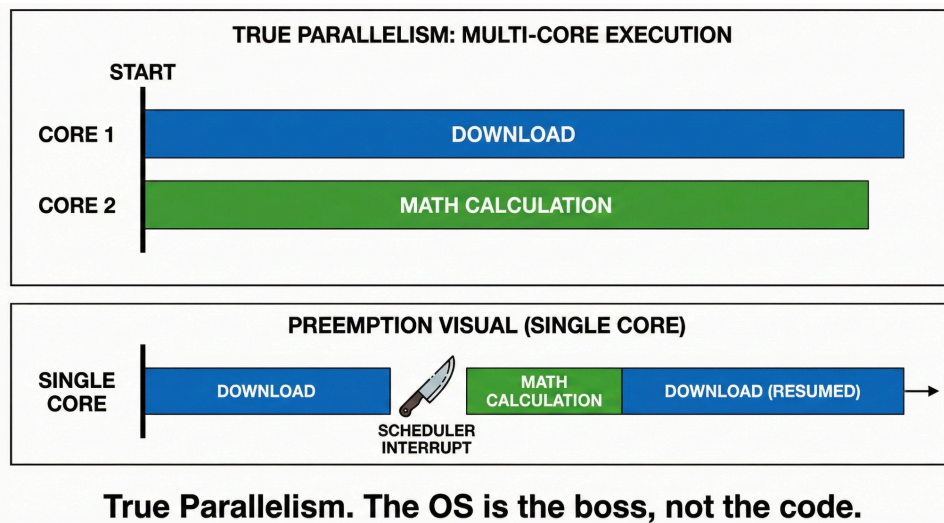
```

Processing A.txt

All tasks completed.

Phase 3: Kernel-Level Preemptive Threading

- **The Standard (1:1):** The model used by most modern OSs (Windows, Linux) for decades.
- **"Preemptive":** The OS forces threads to switch (context switch) so no single thread can hog the CPU.
- **True Parallelism:** Allowed real simultaneous execution on multi-core CPUs.
- **Heavyweight:** Threads became expensive resources (heavy memory usage, slow switching).



Implementation:

```
class Main {  
  
    static void downloadFile(String file) {  
        System.out.println(Thread.currentThread().getName()  
            + " : Downloading " + file);  
  
        try {  
            // Simulate blocking I/O (network / disk)  
            Thread.sleep(3000);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```

    }

    System.out.println(Thread.currentThread().getName()
        + " : Download finished");
}

static void calculateMath() {
    System.out.println(Thread.currentThread().getName()
        + " : Performing heavy calculation");

    long sum = 0;
    for (long i = 0; i < 1_000_000_000L; i++) {
        sum += i;
    }

    System.out.println(Thread.currentThread().getName()
        + " : Calculation completed");
}

public static void main(String[] args) {

    Thread t1 = new Thread(() → {
        downloadFile("A.txt");    // BLOCKS only t1
    }, "Thread-1");

    Thread t2 = new Thread(() → {
        calculateMath();          // Runs in parallel
    }, "Thread-2");

    t1.start();
    t2.start();

}
}

```

Output:

```
Thread-2 : Performing heavy calculation
Thread-1 : Downloading A.txt
Thread-2 : Calculation completed
Thread-1 : Download finished
```

Phase 4: Shift Toward Hybrid Abstractions (Modern Era)

- **Return of M:N:** We are moving back to lighter threads managed by runtimes, not just the OS.
- **Examples:**
 - **Java Virtual Threads:** (As described above).
 - **Go Routines:** (Go Language's lightweight threads).
 - **GCD (Grand Central Dispatch):** Apple's way of managing tasks where the system handles the threads for you.

Virtual Threads

- **Modern M:N Model:** Production-ready many-to-many (hybrid) threading.
- **Lightweight:** Millions of virtual threads possible.
- **Blocking without Blocking:** Blocking calls do **not** block kernel threads.
- **Mechanism:**
 - Run on **Carrier (kernel) threads**
 - **Mount** when executing
 - **Unmount** on blocking
 - Stored in **heap memory** while waiting
- **Importance:** Simplifies high-throughput server design and improves scalability.

Linux & macOS Implementation Details

A. Linux Implementation

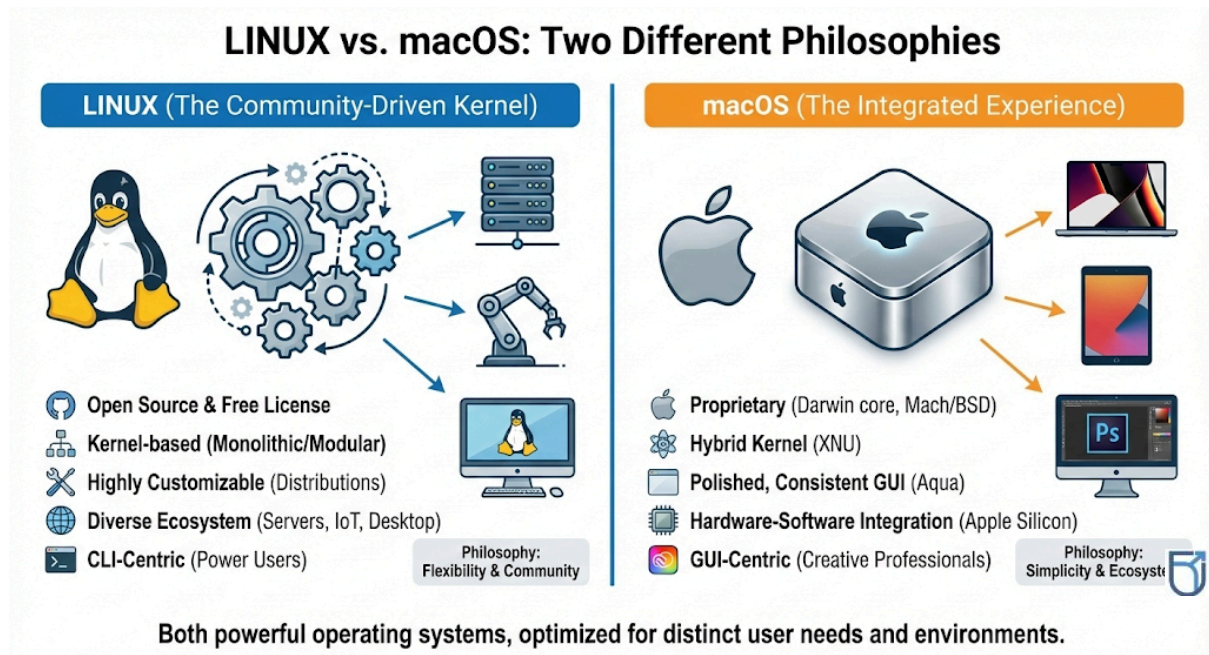
1. The "No Distinction" Philosophy

- **Concept:** Uniquely, the Linux kernel does **not** distinguish between a "process" and a "thread."
- **The Task:** To Linux, everything is just a **Runnable Task**.
- **Data Structure:** Both processes and threads are represented by the exact same data structure called `task_struct`.

2. The `clone()` System Call

- **How it works:** Thread creation is done using the `clone()` system call.

- **Flexibility:** `clone()` takes arguments (flags) that tell the kernel how much to share with the parent.
 - **As a Process:** If you share nothing (copy everything), it behaves like a new process (fork).
 - **As a Thread:** If you share the address space (memory), file system info, and signal handlers, it behaves like a thread.



3. NPTL (Native POSIX Thread Library)

- **The Standard:** This is the modern threading library used in Linux.
- **Model:** It implements a **1:1 Model**.
- **Mapping:** Every single thread you create in your program corresponds to exactly one kernel-scheduled `task_struct` inside the OS.

B. macOS Implementation

1. The XNU Kernel

- **Hybrid Core:** macOS uses the **XNU** kernel, which is a hybrid of the Mach microkernel and BSD (Unix).

2. Mach Threads & Pthreads

- **Low Level:** At the deepest level, the kernel schedules units called **Mach Threads**. These are very low-level and hard to use directly.
- **High Level:** For developers, macOS provides **POSIX Threads (pthreads)**.
- **The Wrapper:** In macOS, every `pthread` is just a "wrapper" around a Mach thread. When you create a pthread, the OS creates a Mach thread behind the scenes.

3. Grand Central Dispatch (GCD)

- **The Modern Way:** Instead of managing threads manually, Apple introduced **GCD**.
- **Task-Based:** Developers just submit "tasks" (blocks of code) to queues.
- **OS Magic:** The OS manages a pool of threads automatically. It decides how many threads to create based on system load, distributing the tasks to the CPU cores efficiently.

Feature	Linux	macOS
Core Concept	No distinction (all are tasks)	Distinct Mach Threads
Main System Call	clone()	Mach API (wrapped by <code>pthread_create</code>)
Threading Model	1:1 (via NPTL)	1:1 (Pthread maps to Mach Thread)
Modern Abstraction	-	Grand Central Dispatch (GCD)

Threading Hazards

1. Race Conditions

Definition:

A situation where the final outcome of a program depends on the unpredictable timing or order of thread execution.

The Cause: Non-Atomic Operations

Commands like `count++` look like one step, but the CPU sees **three separate steps**:

- **Read:** Get the current value of `count` from memory.
- **Modify:** Add 1 to the value.

- **Write:** Save the new value back to memory.

If a thread is interrupted in the middle of these steps, hazards occur.

The Result: Lost Updates

- Two threads try to increment `count` (initially 5) at the same time.
- Both read 5.
- Both add 1 to get 6.
- Both write 6.

2. Deadlock

Definition:

A situation where two or more threads are stuck forever, waiting for each other to release resources.

The Conditions (The “Deadly Embrace”)

- **Resource Holding (Hold and Wait):**
A thread is holding one resource (for example, a file) while waiting to acquire another.
- **Circular Waiting:**
Thread A is waiting for Thread B, and Thread B is waiting for Thread A. A closed loop is formed.

3. Solution: Lock Ordering

The Fix:

To prevent deadlocks, enforce a strict **global order** for acquiring locks.

Rule:

“Always acquire Lock 1 before Lock 2.”

Why it works:

If every thread follows the same order, circular waiting becomes impossible.

Thread B cannot acquire Lock 2 and then wait for Lock 1, because it is not allowed to touch Lock 2 until it already holds Lock 1.

POSIX Threads (pthreads) API

1. Thread Management

- `pthread_create`: Spawns a new thread to run a function.
- `pthread_join`: Blocks and waits for a specific thread to finish (like `wait()`).
- `pthread_exit`: Terminates the current thread safely.

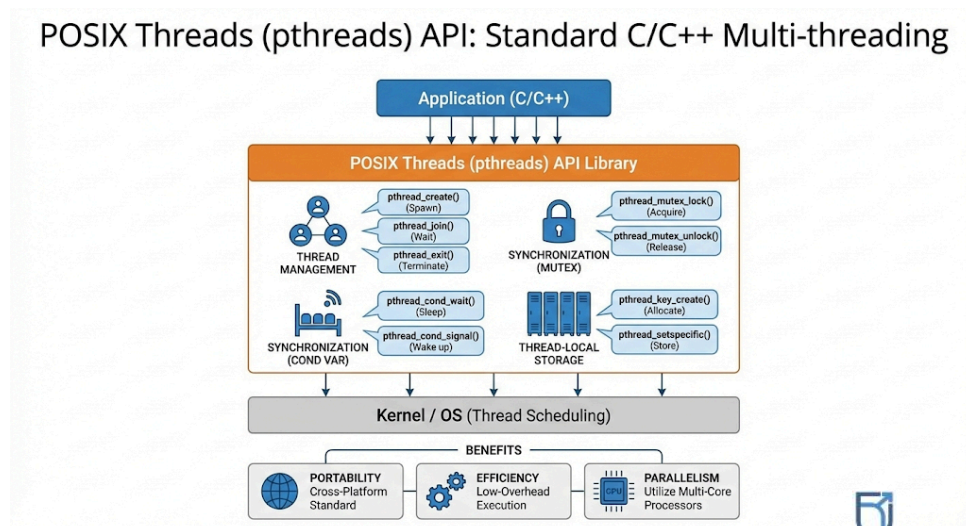
- **pthread_detach**: Runs thread in background; auto-cleans up when done (cannot join).
- **pthread_self**: Returns the ID of the current thread.

2. Attributes (pthread_attr_t)

- **Concept**: Object used to set thread properties *before* creation.
- **Stack Size**: Adjusts memory reserved for the thread.
- **Detached State**: Creates a thread that is already detached (background).
- **Scheduling**: Sets priority rules (e.g., Real-time vs. Standard).

3. Synchronization (Preview)

- **pthread_mutex_t**: Basic lock. Only one thread enters at a time.
- **pthread_cond_t**: Condition variable. Let threads sleep until signaled.
- **pthread_rwlock_t**: Read-Write lock. Allows multiple readers, but only one writer.



Lifecycle Operations

1) Create(pthread_create):

- Creates a new thread
- Starts execution from a given function
- Returns immediately (non-blocking)
- Thread runs concurrently with main thread

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
```

```

void* task(void* arg) {
    int *res = malloc(sizeof(int));
    *res = 10;
    return res;
}

int main() {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, task, NULL);
    pthread_join(t, &ret);

    printf("Thread returned: %d\n", *(int*)ret);
    free(ret);
    return 0;
}

```

② Join (pthread_join)

- Makes **calling thread wait**
- Collects return value of target thread
- Similar to `wait()` for processes
- Prevents **zombie threads**

```

#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

void* task(void* arg) {
    int *res = malloc(sizeof(int));
    *res = 10;
    return res;
}

int main() {
    pthread_t t;
    void *ret;

    pthread_create(&t, NULL, task, NULL);
    pthread_join(t, &ret);

    printf("Thread returned: %d\n", *(int*)ret);
    free(ret);
    return 0;
}

```

③ Detach (pthread_detach)

- Thread runs independently
- Resources freed automatically after termination
- Cannot be joined later
- Useful for background tasks

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* task(void* arg) {
    sleep(2);
    printf("Detached thread finished\n");
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, task, NULL);
    pthread_detach(t);    // no join
    sleep(3);             // allow thread to finish
    return 0;
}
```

4 Termination (**pthread_exit** / **return**)

A thread can terminate by:

1. Returning from start function
2. Calling **pthread_exit()**
3. Being cancelled

```
#include <stdio.h>
#include <pthread.h>

void* task(void* arg) {
    printf("Thread exiting\n");
    pthread_exit("Done");
}

int main() {
    pthread_t t;
    void *msg;

    pthread_create(&t, NULL, task, NULL);
}
```

```
pthread_join(t, &msg);

printf("Thread message: %s\n", (char*)msg);
return 0;
}
```

Language-Level Threading Models

Language	Model	Key Note
C/C++	1:1	Uses Pthreads directly. Fast but heavy.
Java (Modern)	1:1	Maps directly to Kernel threads.
Java (Old)	Green Threads	Used M:1 (User-level only). Obsolete.
Python	1:1 (with GIL)	Has threads, but GIL (Global Interpreter Lock) prevents true CPU parallelism.
Go	M:N	Goroutines . Extremely lightweight and scalable.
Erlang	Lightweight	Uses "Processes" (not OS threads). Built for massive concurrency.
Node.js	Event Loop	Single-threaded Loop + Worker Threads for heavy tasks.